The `UpdateAsync` method uses the `DbContext.Update` method, which tells the context to begin tracking the entity in the `EntityState.Modified` state. The `DbContext .Update` method is also new in EF Core. In previous versions of EF, you had to attach a modified entity to the context and explicitly set its state to `EntityState.Modified`, which is similar to the pattern that has been applied to the existing `OnPostAsync` handler in the scaffolded code:

```
_context.Attach(Property).State = EntityState.Modified;
```

When an entity is in the `Modified` state, EF Core generates an SQL `UPDATE` statement that results in all the entity's non-key values being modified. We will look at how to control the SQL, so it only updates individual property values when you implement the soft delete later.

Both the `CreateAsync` and `UpdateAsync` method include the same line:

```
await _context.SaveChangesAsync();
```

The `SaveChangesAsync` method causes all the pending changes to data to be written to the database. It returns an `int`, representing the number of rows affected by the operation. When you use the `Add` method to create a new record, the generated SQL retrieves the primary key value of the newly created record, and EF Core assigns that to the tracked entity. Your database operations are very simple and only involve one command. It is possible to set up multiple operations and have them all committed at the same time with one call to `SaveChangesAsync`. By default, EF Core